

Compression d'image au moyen de la décomposition en valeurs singulières

adapté d'une présentation de David Richards et Adam Abrahamsen

Mario Lambert¹

¹Faculté des sciences
Université de Sherbrooke

MAT417 – 2 août 2010

PLAN DE LA PRÉSENTATION

1 INTRODUCTION

2 LES CALCULS

3 EXEMPLE

4 COMPRESSION D'IMAGE

5 EXEMPLE

PLAN DE LA PRÉSENTATION

1 INTRODUCTION

2 LES CALCULS

3 EXEMPLE

4 COMPRESSION D'IMAGE

5 EXEMPLE

PLAN DE LA PRÉSENTATION

1 INTRODUCTION

2 LES CALCULS

3 EXEMPLE

4 COMPRESSION D'IMAGE

5 EXEMPLE

PLAN DE LA PRÉSENTATION

- 1 INTRODUCTION
- 2 LES CALCULS
- 3 EXEMPLE
- 4 COMPRESSION D'IMAGE
- 5 EXEMPLE

PLAN DE LA PRÉSENTATION

- 1 INTRODUCTION
- 2 LES CALCULS
- 3 EXEMPLE
- 4 COMPRESSION D'IMAGE
- 5 EXEMPLE

PLAN DE LA PRÉSENTATION

- 1 INTRODUCTION
- 2 LES CALCULS
- 3 EXEMPLE
- 4 COMPRESSION D'IMAGE
- 5 EXEMPLE

INTRODUCTION

La décomposition en valeurs singulières est un processus utilisé dans plusieurs domaines des mathématiques. Dans cette présentation, nous montrons comment utiliser cette technique pour factoriser et approximer de très grandes matrices, qui prendront ici la forme d'images.

LE RÉSULTAT ATTENDU

EN ENTRÉE

$$A \in \mathbf{M}_{m \times n}(\mathbb{R})$$

Si $A \in \mathbf{M}_{m \times n}(\mathbb{C})$, il suffit de remplacer $(\cdot)^t$ par $(\cdot)^*$ dans la suite.

EN SORTIE

$$A = U_{m \times m} S_{m \times n} V_{n \times n}^t \text{ avec}$$

$$\left\{ \begin{array}{l} \bullet U \text{ et } V \text{ des matrices unitaires} \\ \quad \text{c-à-d. } UU^t = U^t U = 1 \\ \quad \text{et } VV^t = V^t V = 1 \\ \bullet S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p) \\ \quad \text{où } p = \min(m, n) \\ \quad r = \text{rg}(A) \\ \quad \text{et } \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \\ \quad \sigma_r > \sigma_{r+1} = \sigma_{r+2} = \dots = \sigma_p = 0 \end{array} \right.$$

LE RÉSULTAT ATTENDU

EN ENTRÉE

$$A \in M_{m \times n}(\mathbb{R})$$

Si $A \in M_{m \times n}(\mathbb{C})$, il suffit de remplacer $(\cdot)^t$ par $(\cdot)^*$ dans la suite.

EN SORTIE

$$A = U_{m \times m} S_{m \times n} V_{n \times n}^t \text{ avec}$$

$$\left\{ \begin{array}{l} \bullet U \text{ et } V \text{ des matrices unitaires} \\ \quad \text{c-à-d. } UU^t = U^t U = 1 \\ \quad \text{et } VV^t = V^t V = 1 \\ \bullet S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p) \\ \quad \text{où } p = \min(m, n) \\ \quad r = \text{rg}(A) \\ \quad \text{et } \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \\ \quad \sigma_r > \sigma_{r+1} = \sigma_{r+2} = \dots = \sigma_p = 0 \end{array} \right.$$

PLAN DE LA PRÉSENTATION

1 INTRODUCTION

2 LES CALCULS

3 EXEMPLE

4 COMPRESSION D'IMAGE

5 EXEMPLE

CALCUL DE V ET S

$$\begin{aligned} A = USV^t &\Rightarrow A^t A = (USV^t)^t (USV^t) \\ &\Rightarrow A^t A = VS^t U^t USV^t \\ &\Rightarrow A^t A = VS^t SV^t \end{aligned}$$

Posons $D = S^t S$. Alors, D est une matrice $n \times n$ qui est *diagonale*.

On a donc $A^t A = VDV^t$, ce qui n'est rien d'autre qu'une diagonalisation de la matrice $A^t A$.

CALCUL DE V ET S (SUITE)

- On calcule les valeurs propres et les vecteurs propres de $A^t A$.
- On place les valeurs propres en ordre décroissant, disons $\lambda_1, \lambda_2, \dots, \lambda_n$.
- On construit la matrice S de taille $m \times n$ avec $S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ où $\sigma_i = \sqrt{\lambda_i}$ et $p = \min(m, n)$.
- On normalise les vecteurs propres de $A^t A$, disons $v_1, v_2, \dots, v_n$.
- On pose $V = (v_1, v_2, \dots, v_n)$.

CALCUL DE V ET S (SUITE)

- On calcule les valeurs propres et les vecteurs propres de $A^t A$.
- On place les valeurs propres en ordre décroissant, disons $\lambda_1, \lambda_2, \dots, \lambda_n$.
- On construit la matrice S de taille $m \times n$ avec $S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ où $\sigma_i = \sqrt{\lambda_i}$ et $p = \min(m, n)$.
- On normalise les vecteurs propres de $A^t A$, disons $v_1, v_2, \dots, v_n$.
- On pose $V = (v_1, v_2, \dots, v_n)$.

CALCUL DE V ET S (SUITE)

- On calcule les valeurs propres et les vecteurs propres de $A^t A$.
- On place les valeurs propres en ordre décroissant, disons $\lambda_1, \lambda_2, \dots, \lambda_n$.
- On construit la matrice S de taille $m \times n$ avec $S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ où $\sigma_i = \sqrt{\lambda_i}$ et $p = \min(m, n)$.
- On normalise les vecteurs propres de $A^t A$, disons $v_1, v_2, \dots, v_n$.
- On pose $V = (v_1, v_2, \dots, v_n)$.

CALCUL DE V ET S (SUITE)

- On calcule les valeurs propres et les vecteurs propres de $A^t A$.
- On place les valeurs propres en ordre décroissant, disons $\lambda_1, \lambda_2, \dots, \lambda_n$.
- On construit la matrice S de taille $m \times n$ avec $S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ où $\sigma_i = \sqrt{\lambda_i}$ et $p = \min(m, n)$.
- On normalise les vecteurs propres de $A^t A$, disons $v_1, v_2, \dots, v_n$.
- On pose $V = (v_1, v_2, \dots, v_n)$.

CALCUL DE V ET S (SUITE)

- On calcule les valeurs propres et les vecteurs propres de $A^t A$.
- On place les valeurs propres en ordre décroissant, disons $\lambda_1, \lambda_2, \dots, \lambda_n$.
- On construit la matrice S de taille $m \times n$ avec $S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ où $\sigma_i = \sqrt{\lambda_i}$ et $p = \min(m, n)$.
- On normalise les vecteurs propres de $A^t A$, disons $v_1, v_2, \dots, v_n$.
- On pose $V = (v_1, v_2, \dots, v_n)$.

CALCUL DE U (ET S)

Pour la matrice U , on a

$$\begin{aligned} A = USV^t &\Rightarrow AA^t = (USV^t)(USV^t)^t \\ &\Rightarrow AA^t = USV^t VS^t U^t \\ &\Rightarrow AA^t = USS^t U^t. \end{aligned}$$

Posons $D' = SS^t$. Alors, D' est une matrice $m \times m$ qui est *diagonale* et qui, en plus, a les « mêmes » éléments diagonaux que la matrice D .

On a donc $AA^t = UD'U^t$, ce qui n'est rien d'autre qu'une diagonalisation de la matrice AA^t .

CALCUL DE U (ET S) (SUITE)

- On calcule les valeurs propres et les vecteurs propres de AA^t .
- On normalise les vecteurs propres de AA^t , disons $u_1, u_2, \dots, u_n$.
- On pose $U = (u_1, u_2, \dots, u_n)$.

CALCUL DE U (ET S) (SUITE)

- On calcule les valeurs propres et les vecteurs propres de AA^t .
- On normalise les vecteurs propres de AA^t , disons $u_1, u_2, \dots, u_n$.
- On pose $U = (u_1, u_2, \dots, u_n)$.

CALCUL DE U (ET S) (SUITE)

- On calcule les valeurs propres et les vecteurs propres de AA^t .
- On normalise les vecteurs propres de AA^t , disons $u_1, u_2, \dots, u_n$.
- On pose $U = (u_1, u_2, \dots, u_n)$.

PLAN DE LA PRÉSENTATION

- 1 INTRODUCTION
- 2 LES CALCULS
- 3 EXEMPLE**
- 4 COMPRESSION D'IMAGE
- 5 EXEMPLE

DONNÉES

On veut construire la décomposition en valeurs singulières de

$$A = \begin{pmatrix} 12 & 18 & 3 \\ 7 & 919 & 122 \end{pmatrix}.$$

CALCUL DE $A^t A$

On a

$$A^t A = \begin{pmatrix} 193 & 6649 & 890 \\ 6649 & 844885 & 112172 \\ 890 & 112172 & 14893 \end{pmatrix}.$$

CALCUL DE S

Les valeurs propres en ordre décroissant de $A^t A$ sont $\lambda_1 \approx 859829,9830$, $\lambda_2 \approx 141,0170$ et $\lambda_3 = 0$.

On construit la matrice S de taille 2×3 avec $S = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_p)$ où $\sigma_i = \sqrt{\lambda_i}$ et $p = \min(m, n)$.

On a

$$S = \begin{pmatrix} 927,2701780 & 0 & 0 \\ 0 & 11,87505789 & 0 \end{pmatrix}.$$

CALCUL DE V

On calcule les vecteurs propres de $A^t A$ puis on les normalise et on les place dans le même ordre que les valeurs propres. On a

$$V = \begin{pmatrix} 0,007803401843 & 0,9986708580 & 0,05094727495 \\ 0,9912712255 & -0,01443091374 & 0,1310461992 \\ 0,1316072359 & 0,04947996357 & -0,9900662950 \end{pmatrix}.$$

CALCUL DE AA^t

Pour calculer U , on passe d'abord par le calcul de

$$AA^t = \begin{pmatrix} 477 & 16992 \\ 16992 & 859494 \end{pmatrix}.$$

CALCUL DE S

Les valeurs propres en ordre décroissant de AA^t sont $\lambda_1 \approx 859829,9830$ et $\lambda_2 \approx 141,0170$ (les mêmes que pour A^tA).

On pose $\sigma_i = \sqrt{\lambda_i}$ pour $i = 1, 2$, puis on a la même matrice 2×3 que précédemment, c'est-à-dire

$$S = \begin{pmatrix} 927,2701780 & 0 & 0 \\ 0 & 11,87505789 & 0 \end{pmatrix}.$$

CALCUL DE U

On calcule les vecteurs propres de AA^t puis on les normalise et on les place dans le même ordre que les valeurs propres. On a

$$U = \begin{pmatrix} 0,01976915146 & 0,9998045712 \\ 0,9998045715 & -0,01976914578 \end{pmatrix}.$$

LE RÉSULTAT

On a

$$A = USV^t$$

$$\approx \begin{pmatrix} 0,01977 & 0,9998 \\ 0,9998 & -0,01977 \end{pmatrix} \begin{pmatrix} 927,3 & 0 & 0 \\ 0 & 11,88 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0,007803 & 0,9987 & 0,05095 \\ 0,9913 & -0,01443 & 0,1310 \\ 0,1316 & 0,04948 & -0,9901 \end{pmatrix}^t$$

PLAN DE LA PRÉSENTATION

- 1 INTRODUCTION
- 2 LES CALCULS
- 3 EXEMPLE
- 4 COMPRESSION D'IMAGE**
- 5 EXEMPLE

LA BASE

- Le but est d'approximer une image (une matrice $m \times n$) en utilisant le moins d'information possible.
- Supposons que $A = USV^t$ avec $u_1, u_2, \dots, u_m$ et $v_1, v_2, \dots, v_n$ les colonnes respectives de U et V et $\sigma_1, \sigma_2, \dots, \sigma_p$ les éléments diagonaux de S .

- On a $A = USV^t \Rightarrow A =$

$$(u_1 \ u_2 \ \dots \ u_m) \begin{pmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & \ddots & \\ & & & & \sigma_p \end{pmatrix} (v_1 \ v_2 \ \dots \ v_n)^t \text{ où}$$

$$r = \text{rg } A.$$

LA BASE

- Le but est d'approximer une image (une matrice $m \times n$) en utilisant le moins d'information possible.
- Supposons que $A = USV^t$ avec $u_1, u_2, \dots, u_m$ et $v_1, v_2, \dots, v_n$ les colonnes respectives de U et V et $\sigma_1, \sigma_2, \dots, \sigma_p$ les éléments diagonaux de S .

- On a $A = USV^t \Rightarrow A =$

$$(u_1 \ u_2 \ \dots \ u_m) \begin{pmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & \ddots & \\ & & & & \sigma_p \end{pmatrix} (v_1 \ v_2 \ \dots \ v_n)^t \text{ où}$$

$$r = \text{rg } A.$$

LA BASE

- Le but est d'approximer une image (une matrice $m \times n$) en utilisant le moins d'information possible.
- Supposons que $A = USV^t$ avec $u_1, u_2, \dots, u_m$ et $v_1, v_2, \dots, v_n$ les colonnes respectives de U et V et $\sigma_1, \sigma_2, \dots, \sigma_p$ les éléments diagonaux de S .

- On a $A = USV^t \Rightarrow A =$

$$(u_1 \ u_2 \ \dots \ u_m) \begin{pmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & \ddots & \\ & & & & \sigma_p \end{pmatrix} (v_1 \ v_2 \ \dots \ v_n)^t \text{ où}$$

$$r = \text{rg } A.$$

LA BASE (SUITE)

Ainsi,

$$A = (u_1 \ u_2 \ \dots \ u_m) \begin{pmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & \ddots & \\ & & & & \sigma_p \end{pmatrix} \begin{pmatrix} v_1^t \\ v_2^t \\ \vdots \\ v_n^t \end{pmatrix}$$

d'où on tire

$$A = \sigma_1 u_1 v_1^t + \sigma_2 u_2 v_2^t + \dots + \sigma_r u_r v_r^t + \dots + \sigma_p u_p v_p^t.$$

LA BASE (SUITE)

Comme les valeurs singulières de A sont toutes positives ou nulles, le fait d'ajouter à la somme précédente des termes nuls est plus qu'inutile. On peut donc réduire la somme à

$$A = \sigma_1 u_1 v_1^t + \sigma_2 u_2 v_2^t + \dots + \sigma_r u_r v_r^t.$$

De plus, comme les valeurs singulières sont en ordre décroissant, il suffit de laisser tomber un certain nombre de termes à la fin de la somme pour diminuer la quantité d'information nécessaire à stocker, tout en conservant une qualité d'image raisonnable.

PLAN DE LA PRÉSENTATION

- 1 INTRODUCTION
- 2 LES CALCULS
- 3 EXEMPLE
- 4 COMPRESSION D'IMAGE
- 5 EXEMPLE**

EXEMPLE

Cliquez sur le lien ci-dessous pour voir Lena en exemple

LenaSVD



Using Matlab to perform (SVD)

- Matlab provides us with the ability to perform (SVD) on larger matrices. For an example we will use a 7×7 matrix with rank 5.
- Using the command

```
A=randint(7,7,25,5)+25
```

provides us with a 7×7 matrix (A) with values between 0 and 50, and with a rank of 5.



$$A = \begin{pmatrix} 24 & 36 & 31 & 31 & 39 & 44 & 39 \\ 27 & 39 & 35 & 33 & 33 & 35 & 33 \\ 29 & 37 & 35 & 43 & 35 & 33 & 35 \\ 32 & 29 & 43 & 36 & 27 & 35 & 27 \\ 39 & 35 & 30 & 31 & 25 & 15 & 25 \\ 29 & 44 & 26 & 47 & 40 & 26 & 40 \\ 34 & 21 & 37 & 22 & 27 & 39 & 27 \end{pmatrix}$$

- Now we can use the power of matlab again to perform (SVD) on the matrix (A). Using the commands:

$[U,S,V]=\text{svd}(A)$ factors A into USV^T .



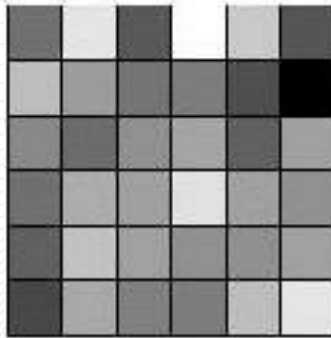


- After factoring (A), using the command

```
svdimage(A,U,S,V,1,gray)
```

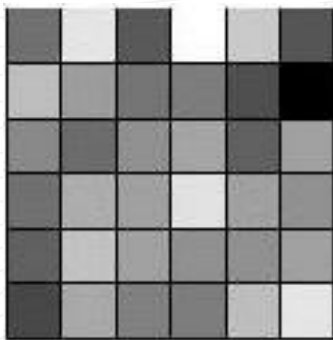
gives us the opportunity to view the matrix A as an image, and see each individual iteration of (A) using the rank approximation. A little explanation of the entries of this function require some explanation. The A , U , S , and V are self explanatory, but the 1 and the gray haven't been explained yet. The "1" starts the program off with the first iteration and the "gray" uses the colormap that matlab defines as gray. Now each number of the matrix corresponds to the color that matlab has defined as the colormap(gray).



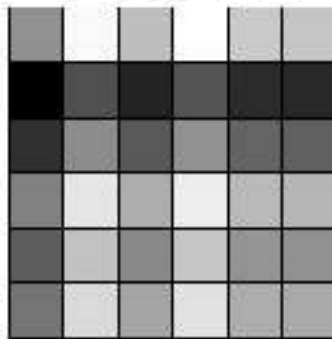


Original Matrix



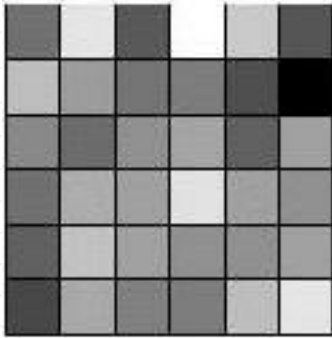


Original Matrix

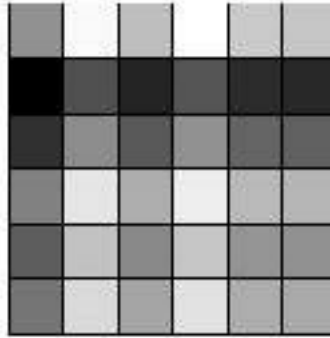


1 Iteration

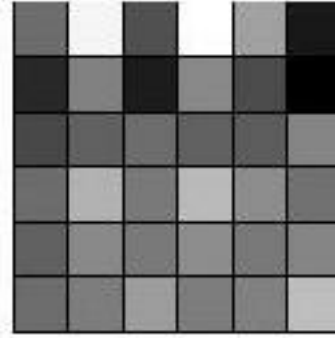




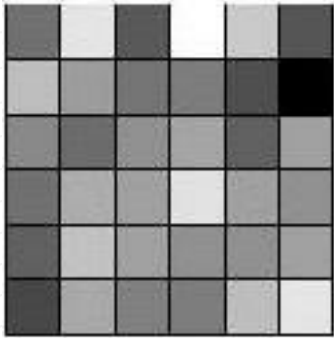
Original Matrix



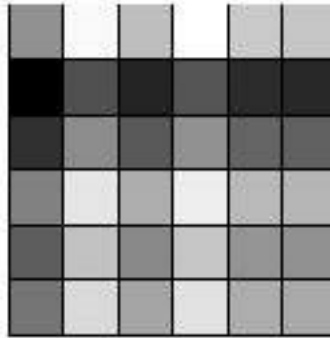
1 Iteration



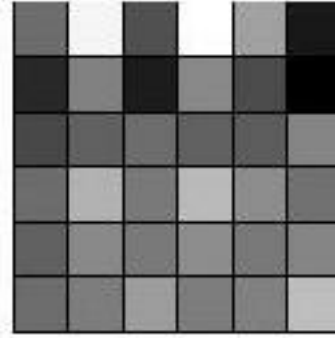
2 Iterations



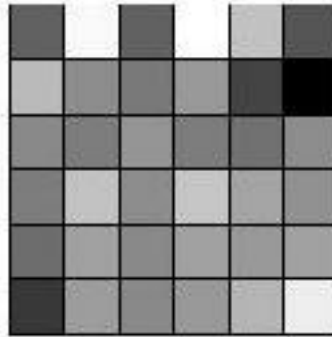
Original Matrix



1 Iteration

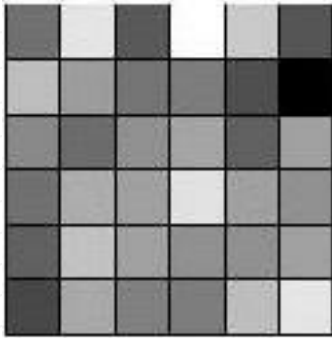


2 Iterations

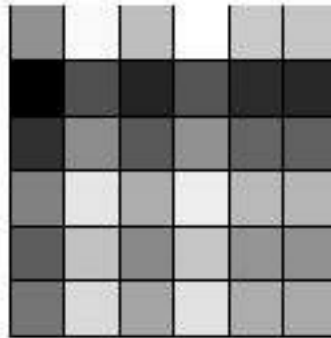


3 Iterations

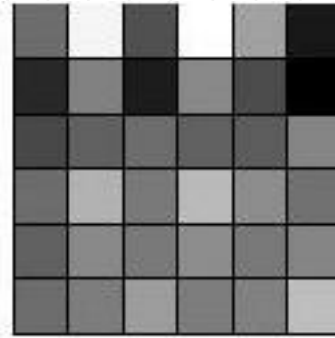




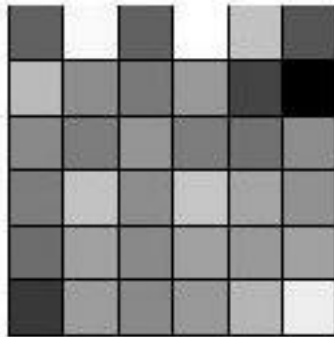
Original Matrix



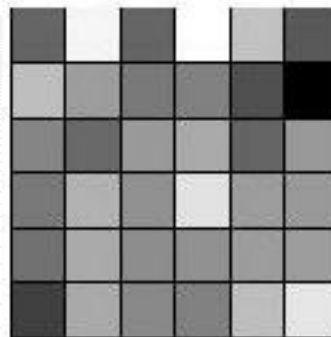
1 Iteration



2 Iterations

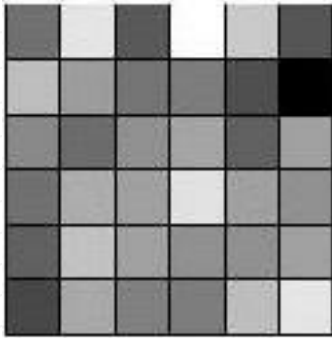


3 Iterations

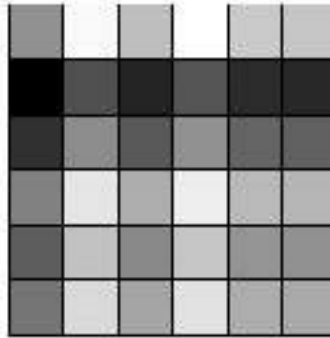


4 Iterations

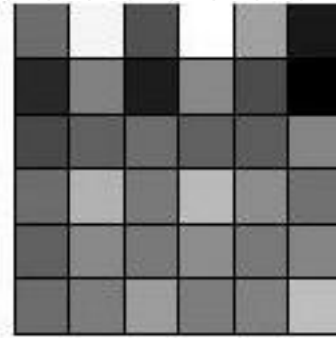




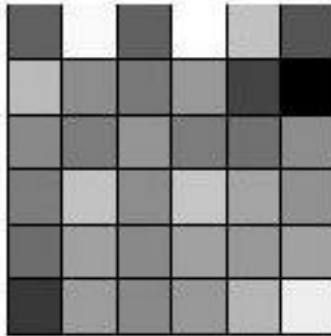
Original Matrix



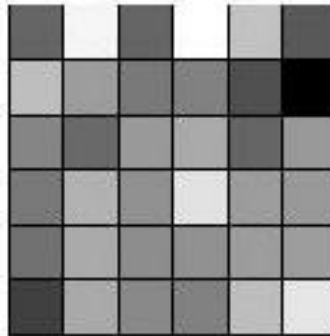
1 Iteration



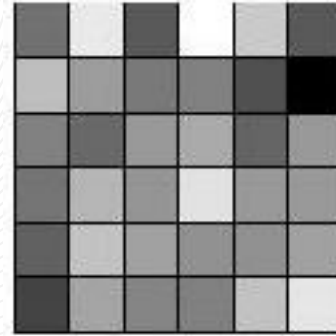
2 Iterations



3 Iterations



4 Iterations

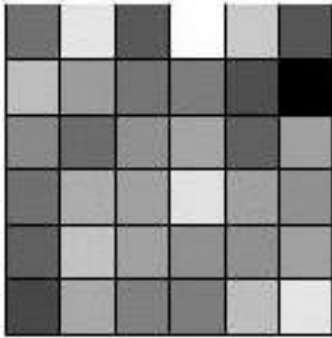


5 Iterations

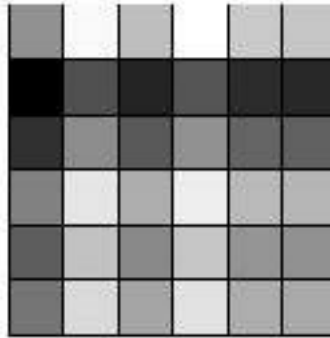


- As you can see here, five iterations gives us the exact image back.

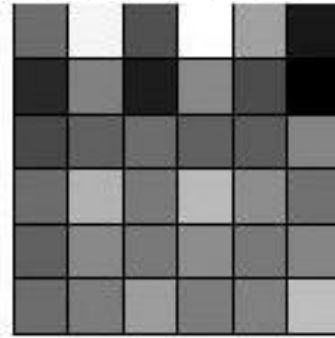




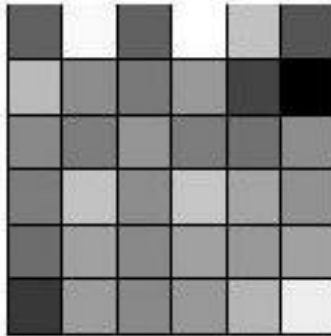
Original Matrix



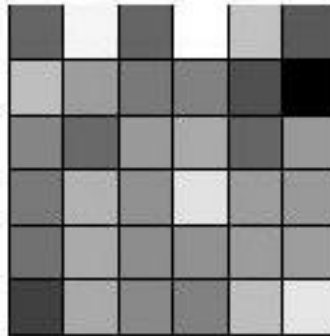
1 Iteration



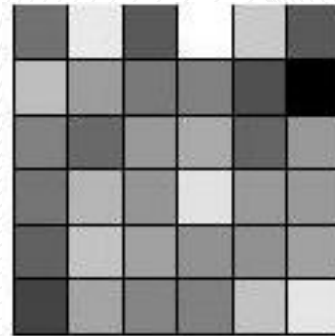
2 Iterations



3 Iterations



4 Iterations



5 Iterations



- As you can see here, five iterations gives us the exact image back.
- So as we mentioned earlier the dependent singular values are zero after we exceed the rank of the matrix. That is why after five iterations (the rank of this matrix) we get an exact approximation.
- For the pictures in this presentation we used a program that we wrote with the help of David Arnold. We had to write this program in order to fit these images into the document. Using the commands,





```
close all
[A,map]=imread('lena.gif');
B=im2double(A,'indexed');
imshow(B,map)
[u,s,v]=svd(B);
C=zeros(size(B));
    for j=1:k
        C=C+s(j,j)*u(:,j)*v(:,j).';
    end
C=floor(C);
imshow(C,map)
k=find(C<1);
C(k)=1;
set(gcf,'Unit','inches','Paperposition',[0,0,2,1])
print -djpeg 'lenak.jpg'
```





- By changing the k values in the forloop we were able to construct the different iterations. We also used this for the previous block images but with a few changes. You may notice the similarities between these two equations:

$$C = C + s(j, j) * u(:, j) * v(:, j) . '$$

and

$$A = \sigma_1 u_1 v_1^T + r \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T$$

- Now we will show the use of (SVD) on Lena which is one of the standardized images.





Original Image



27/41





Original Image



5 Iterations





Original Image



5 Iterations



10 Iterations





Original Image



5 Iterations



10 Iterations



20 Iterations





Original Image



5 Iterations



10 Iterations



20 Iterations



60 Iterations





Original Image



5 Iterations



10 Iterations



20 Iterations



60 Iterations



100 Iterations





- The first five iterations actually give the shape of the image, which is a decent approximation. This takes up 99.9% less storage space than the original image.





Original Image



5 Iterations



10 Iterations



20 Iterations



60 Iterations



100 Iterations





- The first five iterations actually give the shape of the image, which is a decent approximation. This takes up 99.9% less storage space than the original image.
- Using the first sixty iterations we get a good approximation, we can identify the person with a substantial degree of detail. This image requires 84% less storage space than the original image. This is good.





Original Image



5 Iterations



10 Iterations



20 Iterations



60 Iterations



100 Iterations





- The first five iterations actually give the shape of the image, which is a decent approximation. This takes up 99.9% less storage space than the original image.
- Using the first sixty iterations we get a good approximation, we can identify the person with a substantial degree of detail. This image requires 84% less storage space than the original image. This is good.
- Finally, the first one-hundred iterations give a near perfect image, and yet requires 55% less storage space. This is very good!





Original Image



5 Iterations



10 Iterations



20 Iterations



60 Iterations



100 Iterations





- The first five iterations actually give the shape of the image, which is a decent approximation. This takes up 99.9% less storage space than the original image.
- Using the first sixty iterations we get a good approximation, we can identify the person with a substantial degree of detail. This image requires 84% less storage space than the original image. This is good.
- Finally, the first one-hundred iterations give a near perfect image, and yet requires 55% less storage space. This is very good!
- Using (SVD) for image compression can be a very useful tool to save storage space. We were able to get an image that is indistinguishable from the original image, but only using 45% of the original storage space.



POUR EN SAVOIR PLUS I



David Richards et Adam Abrahamsen

Image Compression using Singular Value Decomposition.



Dan Kalman

A Singularly Valuable Decomposition.

The College Mathematics Journal, vol. 27, no. 1, Janvier 1998, pp. 2-23.

